



PROJECT BLUEPRINT

TechyGuide

Visual Programming Platform for ESP32 & IoT Education

Document	Project Blueprint & Technical Specification
Version	1.0.0
Date	June 23, 2026
Status	Phase 1 — Delivered (40% Complete)
Classification	Technical Specification & Development Roadmap

A web-based visual programming environment combining Scratch-style animation with ESP32 hardware programming for IoT education and rapid prototyping.

Table of Contents

- 1. Executive Summary
- 2. Project Overview
- 3. Technology Stack
- 4. System Architecture
- 5. Feature Inventory
- 6. Block Specification
- 7. Code Generation Pipeline
- 8. Current Delivery — Phase 1
- 9. 12-Month Roadmap
- 10. Milestone Deliverables
- 11. Quality Assurance & Testing
- 12. Risk Assessment
- 13. Appendix

1. Executive Summary

TechyGuide is an enterprise-grade, web-based visual programming platform designed for ESP32 microcontroller education and IoT prototyping. The platform merges two paradigms into a unified workspace:

- **Scratch Mode** — Drag-and-drop block programming with animated sprites, stage rendering, and event-driven logic, inspired by MIT Scratch.
- **Board Mode** — ESP32-specific hardware blocks spanning GPIO control, sensor integration, actuator management, display output, motor control, and full IoT connectivity.

The platform generates production-ready **MicroPython** and **Arduino C++** code from visual blocks, and supports direct code upload to ESP32 hardware via the **Web Serial API** and **Arduino CLI** — all from within the browser, with zero driver installation required.

The full-featured platform — including AI/ML integration, multi-board support (ESP32, Raspberry Pi), cloud database, and advanced IoT features — is estimated to take **approximately 12 months** to complete across **6 development phases**.

Key Metrics

Metric	Value
Total Block Types	~317
ESP32 Hardware Blocks	167 (across 8 categories)
Scratch Animation Blocks	96 (across 7 categories)
Standard Programming Blocks	54 (across 6 categories)
Code Generator Files	47
UI Components	14
Supported Languages	MicroPython, Arduino C++, Python (RPi)
Target Hardware	ESP32, ESP32-S2/S3/C3, ESP32-CAM, Raspberry Pi Pico, Raspberry Pi 4/5
Planned Integrations	AI/ML (TensorFlow Lite), Cloud Database, Real-time Dashboard
Estimated Development	~12 months (6 phases)
Current Status	Phase 1 Delivered — 40%

2. Project Overview

2.1 Problem Statement

Current IoT education tools force learners to choose between visual simplicity (Scratch) and hardware capability (Arduino IDE). There is no unified platform that offers both animated visual feedback *and* real hardware programming in a single browser-based environment.

2.2 Solution

TechyGuide bridges this gap by providing:

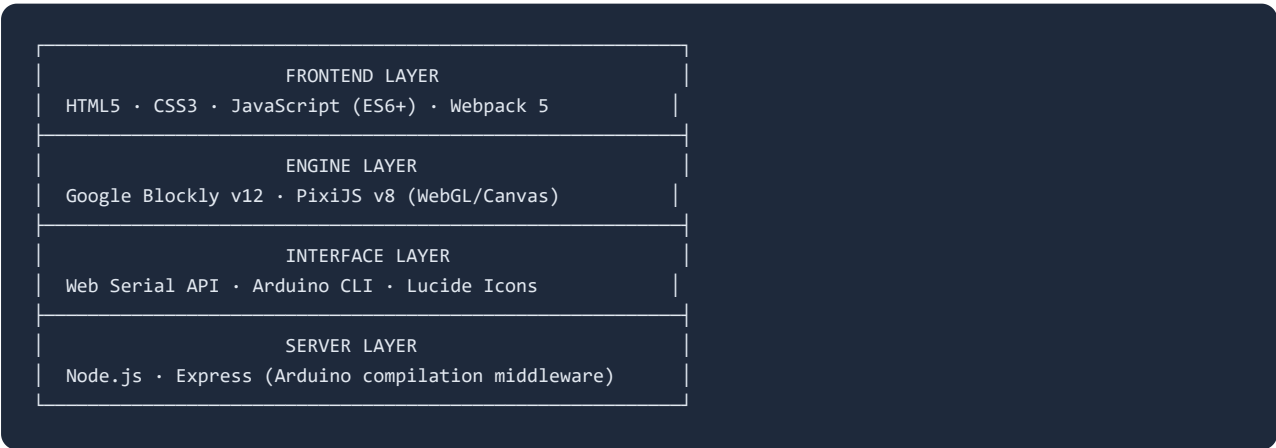
- 1. **A dual-mode workspace** — seamless switching between Scratch animation and ESP32 hardware programming.
- 2. **Dual code generation** — every block program produces both MicroPython and Arduino C++ output.
- 3. **Browser-native upload** — code is compiled and flashed to ESP32 directly from Chrome/Edge using Web Serial API, with no additional software installation.
- 4. **Multi-board support** — planned support for ESP32 variants (S2, S3, C3), Raspberry Pi Pico (RP2040), and Raspberry Pi 4/5 single-board computers.
- 5. **Comprehensive block library** — 317+ block types covering everything from basic LED control to MQTT messaging, Blynk IoT integration, and ESP32-CAM camera capture.
- 6. **AI/ML integration** — planned TensorFlow Lite Micro support for on-device machine learning (image classification, anomaly detection, voice recognition).
- 7. **Cloud database** — planned Firebase/Supabase integration for real-time data storage, user authentication, and project cloud sync.

2.3 Target Users

Segment	Use Case
K-12 Students	Learn programming fundamentals via animated sprites
University Students	Hardware programming, sensor integration, IoT protocols
Educators & Trainers	Curriculum delivery with visual + hardware exercises
Hobbyists & Makers	Rapid prototyping of ESP32 IoT projects
EdTech Institutions	Licensed deployment for classroom and lab environments

3. Technology Stack

3.1 Stack Overview



3.2 Component Detail

Layer	Technology	Version	Purpose
Block Engine	Google Blockly	v12	Visual block workspace, toolbox, code generation framework
Graphics Engine	PixiJS	v8	WebGL/Canvas sprite rendering, stage animation, visual effects
Build System	Webpack	5	Module bundling, asset pipeline, code splitting, HMR
Upload — Browser	Web Serial API	Native	Direct USB serial communication (Chrome 89+, Edge 89+)
Upload — Server	Arduino CLI	Latest	Server-side compilation and upload for Arduino C++
Icons	Lucide	Latest	Consistent, lightweight SVG icon system
Server	Node.js + Express	LTS	Arduino compilation middleware, project API
Frontend	HTML5 / CSS3 / JS	ES6+	Application shell, UI components, state management

3.3 Browser Compatibility

Browser	Minimum Version	Web Serial API	Status
Google Chrome	89+	✔ Supported	Primary Target
Microsoft Edge	89+	✔ Supported	Primary Target
Opera	75+	✔ Supported	Secondary
Firefox	—	✘ Not Supported	Not Supported
Safari	—	✘ Not Supported	Not Supported

4. System Architecture

4.1 High-Level Architecture

```

flowchart TB
    subgraph USER["👤 User"]
        A["Browser (Chrome/Edge)"]
    end

    subgraph WORKSPACE["🧩 Blockly Workspace"]
        B["Scratch Mode\n96 Animation Blocks"]
        C["Board Mode\n167 ESP32 Blocks"]
        D["Standard Blocks\n54 Programming Blocks"]
    end

    subgraph GENERATORS["⚙️ Code Generators"]
        E["MicroPython\nGenerator"]
        F["Arduino C++\nGenerator"]
    end

    subgraph UPLOAD["📤 Upload Pipeline"]
        G["Web Serial API\n(Browser-native)"]
        H["Arduino CLI\n(Server-side)"]
    end

    subgraph SCRATCH_ENGINE["🎮 Scratch Engine"]
        I["Block Interpreter"]
        J["PixiJS Stage\n(WebGL/Canvas)"]
        K["Sprite Engine"]
    end

    subgraph HARDWARE["🔌 Hardware"]
        L["ESP32\nMicrocontroller"]
    end

    A --> B
    A --> C
    A --> D

    C --> E
    C --> F
    D --> E
    D --> F

    E -->|"MicroPython\n(.py)"| G
    F -->|"Arduino C++\n(.ino)"| H

    G -->|"USB Serial"| L
    H -->|"Compile & Flash"| L

    B --> I
    I --> J
    J --> K
  
```

4.2 Upload Pipeline Architecture

```
flowchart LR
    subgraph MICROPYTHON["MicroPython Path"]
        MP1["Block Program"] --> MP2["MicroPython\nCode Generator"]
        MP2 --> MP3[".py Source File"]
        MP3 --> MP4["Web Serial API"]
        MP4 --> MP5["ESP32\n(MicroPython Firmware)"]
    end

    subgraph ARDUINO["Arduino C++ Path"]
        AR1["Block Program"] --> AR2["Arduino C++\nCode Generator"]
        AR2 --> AR3[".ino Source File"]
        AR3 --> AR4["Express Server"]
        AR4 --> AR5["Arduino CLI\n(Compile)"]
        AR5 --> AR6[".bin Firmware"]
        AR6 --> AR7["Upload to ESP32"]
    end
```

4.3 Scratch Engine Architecture

```
flowchart TB
    subgraph ENGINE["Scratch Execution Engine"]
        A["Event Bus"] -->|green flag / key press / click| B["Block Interpreter"]
        B -->|motion, looks, sound commands| C["Sprite Engine"]
        C -->|position, costume, effects| D["Sprite Store"]
        D -->|render state| E["Stage Renderer\n(PixiJS v8)"]
        E -->|WebGL / Canvas| F["Visual Output\n(480 x 360 Stage)"]
    end
```

5. Feature Inventory

5.1 Complete Feature Matrix

#	Feature	Component Count	Status (Phase 1)
1	ESP32 Hardware Blocks	167 block types	32 blocks enabled (19%)
2	Standard Programming Blocks	54 block types	All 54 enabled (100%)
3	Scratch Animation Blocks	96 block types	All 96 enabled (100%)
4	MicroPython Code Generators	19 generator files	Partial
5	Arduino C++ Code Generators	19 generator files	Phase 3
6	Core Framework Generators	9 generator files	Complete
7	Block Interpreter Engine	1 file	Complete
8	Stage Renderer (PixiJS)	1 file	Complete

#	Feature	Component Count	Status (Phase 1)
9	Sprite Engine	1 file	● Complete
10	Sprite Store	1 file	● Complete
11	Event Bus	1 file	● Complete
12	UI Components	14 components	● Core set enabled
13	Serial Monitor	1 component	● Read-only
14	Upload System	2 paths	● Phase 2–3

Legend: ● Complete — ● Partial — ● Planned

5.2 UI Component Inventory

#	Component	Description	Phase
1	Mode Switcher	Toggle between Scratch Mode and Board Mode	1
2	Block Search	Search and filter blocks across categories	1
3	Theme Support	Light/dark theme switching	1
4	Project Save/Load	Local project persistence (JSON export/import)	1
5	Serial Monitor	Real-time serial communication with ESP32	2
6	Connect Modal	USB device selection and connection management	2
7	Upload Panel	Code upload interface with progress indicators	2–3
8	Sprite Panel	Sprite management (add, delete, duplicate, rename)	1
9	Sprite Library	Pre-built sprite asset browser	1
10	Backdrop Library	Stage backdrop selection and management	1
11	Library Manager	Installable component/sensor libraries	5
12	Subscription Modal	License tier selection and management	6
13	Landing Page	Product landing page with feature showcase	1
14	Code Preview Panel	Generated code display with syntax highlighting	1

6. Block Specification

6.1 ESP32 Hardware Blocks — 167 Types

#	Category	Subcategory	Block Count	Description
1	ESP32 Core	Program + Pins	11	Setup/loop structure, digital/analog read-write, PWM, pin modes
2	Inputs	Tactile, Slide, Touch & Hall	9	Push buttons, slide switches, capacitive touch, Hall effect
3	Sensors	Temperature	7	DHT11/22 temperature and humidity
4		Ultrasonic	4	HC-SR04 distance measurement
5		Environmental	4	BMP280 pressure/altitude, rain sensor
6		Motion	4	MPU6050 accelerometer/gyroscope
7		RFID	3	RC522 RFID reader/writer
8		Infrared	4	IR receiver, IR remote decode
9		Fire Detection	3	Flame sensor, smoke detection
10		Water	3	Water level, moisture sensor
11		Sound	3	Sound sensor, microphone module
12		Light	4	LDR, BH1750 lux meter
13		Gas	3	MQ-series gas sensors
14		Soil	3	Soil moisture, soil temperature
15		Color	2	TCS3200 color sensor
16		Sensors Subtotal		47
17	Actuators	Servo Motor	3	Angle control, sweep, detach
18		Relay	2	Single/multi-channel relay control
19		LED	3	Digital LED, RGB LED, fade/blink
20		Buzzer	3	Tone, melody, frequency control
21		Pump	2	Water pump on/off, speed control
22		Notification	3	Piezo alert, visual indicator
23		Music	2	Note playback, melody sequences
24		Actuators Subtotal		18
25	Displays	I2C LCD 16×2	4	Print, clear, set cursor, backlight
26		NeoPixel WS2812B	3	Set color, fill, rainbow effects

#	Category	Subcategory	Block Count	Description
27		Displays Subtotal		7
28	Motors	L298N Driver	5	Forward, reverse, speed, stop, brake
29		Generic DC	3	Simple DC motor control
30		Motors Subtotal		8
31	Comms & IoT	Serial	4	Print, read, baudrate, formatting
32		Bluetooth	6	BLE init, advertise, send, receive, scan, pair
33		WiFi	8	Connect, AP mode, scan, status, IP, disconnect, reconnect, signal
34		HTTP	8	GET, POST, PUT, DELETE, headers, JSON parse, response, URL encode
35		MQTT	10	Connect, publish, subscribe, callback, QoS, retain, LWT, disconnect, reconnect, status
36		Blynk IoT	8	Auth, virtual write, virtual read, notifications, events, datastreams, sync, connect
37		ThingSpeak	6	API key, write field, read field, channel, update interval, status
38		Camera (ESP32-CAM)	6	Init, capture, resolution, stream, save, quality
39		Storage (SPIFFS/SD)	4	File write, file read, list files, delete
40		Comms & IoT Subtotal		60
41	Dabble App	Gamepad	3	Button state, joystick X/Y
42		Phone Sensors	2	Accelerometer, gyroscope data
43		Color Detector	2	Camera color detection, RGB values
44		Dabble Subtotal		7
		ESP32 TOTAL		167

6.2 Standard Programming Blocks — 54 Types

Category	Block Count	Examples
Logic	9	if/else, comparison, and/or/not, boolean, ternary
Loops	8	repeat, while, for, for-each, break, continue
Math	12	arithmetic, trigonometry, rounding, random, constrain, map
Text	10	create, join, length, indexOf, charAt, substring, trim, case
Lists	9	create, get, set, insert, remove, length, indexOf, sort

Category	Block Count	Examples
Variables	3	create, set, change
Functions	3	define (no return), define (with return), call
Total	54	

6.3 Scratch Animation Blocks — 96 Types

Category	Block Count	Examples
Motion	18	move, turn, go to, glide, point, position, direction, bounce
Looks	16	say, think, switch costume, show/hide, size, graphic effects
Sound	10	play sound, stop, volume, pitch, audio effects
Events	12	green flag, key pressed, sprite clicked, broadcast, receive
Control	14	wait, repeat, forever, if/else, stop, clone, delete clone
Sensing	14	touching, distance, ask, answer, mouse, keyboard, timer
Operators	12	arithmetic, comparison, logic, random, string ops, modulo
Total	96	

6.4 Block Count Summary

Category	Count	% of Total
ESP32 Hardware Blocks	167	52.7%
Scratch Animation Blocks	96	30.3%
Standard Programming Blocks	54	17.0%
Grand Total	317	100%

7. Code Generation Pipeline

7.1 Generator File Distribution

#	Generator Category	File Count	Target Language	Phase
1	Core Framework	9	Both	1 
2	ESP32 Core + Pins	1 + 1	MicroPython	1 
3	ESP32 Core + Pins	1 + 1	Arduino C++	3

#	Generator Category	File Count	Target Language	Phase
4	Inputs	1	MicroPython	1 
5	Inputs	1	Arduino C++	3
6	Sensors	1	MicroPython	2
7	Sensors	1	Arduino C++	3
8	Actuators	1	MicroPython	2
9	Actuators	1	Arduino C++	3
10	Displays	1	MicroPython	4
11	Displays	1	Arduino C++	4
12	Motors	1	MicroPython	4
13	Motors	1	Arduino C++	4
14	Comms & IoT	1	MicroPython	5
15	Comms & IoT	1	Arduino C++	5
16	Dabble	1	MicroPython	6
17	Dabble	1	Arduino C++	6
18	Scratch Blocks	1	JavaScript (runtime)	1 
19	Standard Blocks	1	MicroPython	1 
20	Standard Blocks	1	Arduino C++	3
	Total	47		

7.2 Code Generation Example

Visual Blocks → Dual Output:

```
[When Program Starts]
[Set Pin 2 as OUTPUT]
[Forever]
  [Set Pin 2 HIGH]
  [Wait 1 second]
  [Set Pin 2 LOW]
  [Wait 1 second]
```

MicroPython Output:

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)

while True:
    led.value(1)
    time.sleep(1)
    led.value(0)
    time.sleep(1)
```

Arduino C++ Output:

```
void setup() {
    pinMode(2, OUTPUT);
}

void loop() {
    digitalWrite(2, HIGH);
    delay(1000);
    digitalWrite(2, LOW);
    delay(1000);
}
```

8. Current Delivery — Phase 1

8.1 Phase 1 Completion Summary

Delivery Date: June 2026 **Completion:** 40% of total project scope

Subsystem	Scope	Delivered	Status
Scratch Mode	96 blocks	96 / 96	✔ 100%
Standard Blocks	54 blocks	54 / 54	✔ 100%
ESP32 Core (Program + Pins)	11 blocks	11 / 11	✔ 100%
ESP32 Inputs	9 blocks	9 / 9	✔ 100%
ESP32 Sensors (DHT only)	47 blocks	7 / 47	● 15%
ESP32 Actuators (LED/Buzzer)	18 blocks	5 / 18	● 28%
ESP32 Displays	7 blocks	0 / 7	☐ Phase 4
ESP32 Motors	8 blocks	0 / 8	☐ Phase 4
ESP32 Comms & IoT	60 blocks	0 / 60	☐ Phase 5
ESP32 Dabble	7 blocks	0 / 7	☐ Phase 6
MicroPython Generation	19 files	Partial	● In progress

Subsystem	Scope	Delivered	Status
Arduino C++ Generation	19 files	0 / 19	☐ Phase 3
Upload System	2 paths	0 / 2	☐ Phase 2-3
Serial Monitor	Full-featured	Read-only	 Partial
Landing Page	Complete	Complete	 100%
Sprite Engine	Complete	Complete	 100%
Stage Renderer	Complete	Complete	 100%
Block Interpreter	Complete	Complete	 100%
Project Save/Load	Local	Local	 100%
Block Search	Complete	Complete	 100%
Theme Support	Complete	Complete	 100%

8.2 ESP32 Block Enablement — Phase 1

Category	Total	Enabled	Remaining
ESP32 Core	11	11	0
Inputs	9	9	0
Sensors	47	7	40
Actuators	18	5	13
Displays	7	0	7
Motors	8	0	8
Comms & IoT	60	0	60
Dabble	7	0	7
Total	167	32	135

ESP32 Enablement Rate: $32 / 167 = 19.2\%$ **Overall Platform Completion (including Scratch + Standard):** $182 / 317$ blocks = **~40%**

9. 12-Month Roadmap

9.1 Phase Timeline

```
gantt
  title TechyGuide – 12-Month Development Roadmap
  dateFormat YYYY-MM
  axisFormat %b %Y

  section Phase 1
  Foundation (40%)           :done, p1, 2026-06, 2026-06

  section Phase 2
  Sensor Expansion + Upload (52%) :active, p2, 2026-07, 2026-08

  section Phase 3
  Arduino + Actuators (65%)      :p3, 2026-09, 2026-10

  section Phase 4
  Displays + Motors (78%)        :p4, 2026-11, 2026-12

  section Phase 5
  Communication + IoT (90%)      :p5, 2027-01, 2027-03

  section Phase 6
  Multi-Board + AI/ML + DB (100%) :p6, 2027-04, 2027-06
```

9.2 Phase Detail

Phase 1 — Foundation · Month 0 · DELIVERED

Completion: 0% → 40%

Deliverable	Detail
Landing Page	Full product landing page with feature showcase
Scratch Mode	All 96 animation blocks, sprite engine, stage renderer
Standard Blocks	All 54 programming blocks (Logic, Loops, Math, Text, Lists, Variables, Functions)
ESP32 Core	11 blocks — program structure, pin read/write, PWM
ESP32 Inputs	9 blocks — buttons, switches, touch, Hall effect
Basic Sensors	7 blocks — DHT11/22 temperature and humidity
Basic Actuators	5 blocks — LED control, buzzer tones
MicroPython Generation	Code generation for enabled blocks
Serial Monitor	Read-only serial output display
Project Save/Load	Local JSON export and import

Deliverable	Detail
Block Search	Cross-category block search and filtering
Theme Support	Light and dark theme switching

Phase 2 — Sensor Expansion + Upload · Months 1–2

Completion: 40% → 52%

Deliverable	Detail
Full Sensor Library	40 additional blocks — ultrasonic, environmental, motion, RFID, IR, fire, water, sound, light, gas, soil, color
MicroPython Upload	Direct code upload to ESP32 via Web Serial API
Serial Monitor v2	Bidirectional read/write, baud rate selection, timestamps, auto-scroll
Connect Modal	USB device selection, connection state management, auto-reconnect

Phase 3 — Arduino C++ + Actuators · Months 3–4

Completion: 52% → 65%

Deliverable	Detail
Arduino C++ Generators	19 generator files producing valid Arduino C++ for all enabled blocks
Compile & Upload Pipeline	Express server with Arduino CLI integration for server-side compilation
Full Actuator Library	13 additional blocks — servo, relay, pump, notification, music
Dual Code Preview	Side-by-side MicroPython and Arduino C++ code display

Phase 4 — Displays + Motors · Months 5–6

Completion: 65% → 78%

Deliverable	Detail
I2C LCD Display	4 blocks — print, clear, cursor positioning, backlight control
NeoPixel WS2812B	3 blocks — individual LED color, fill, rainbow animation
L298N Motor Driver	5 blocks — forward, reverse, speed control, stop, brake
Generic DC Motor	3 blocks — on/off, speed, direction

Phase 5 — Communication + IoT · Months 7–9

Completion: 78% → 90%

Deliverable	Detail
WiFi	8 blocks — connect, access point, scan, status, IP retrieval
Bluetooth (BLE)	6 blocks — init, advertise, send, receive, scan, pair
HTTP Client	8 blocks — GET, POST, PUT, DELETE, JSON parsing
MQTT	10 blocks — connect, publish, subscribe, QoS, LWT
Blynk IoT	8 blocks — authentication, virtual pins, events, notifications
ThingSpeak	6 blocks — channel write/read, update intervals
Storage (SPIFFS/SD)	4 blocks — file read, write, list, delete
Library Manager	Installable component and sensor library system

Phase 6 — Multi-Board, AI/ML, Database & Polish · Months 10–12

Completion: 90% → 100%

Deliverable	Detail
ESP32-CAM Camera	6 blocks — init, capture, resolution, stream, save, quality
Dabble App Integration	7 blocks — gamepad, phone sensors, color detection
Multi-Board Support	
↳ ESP32 Variants	Board selection for ESP32, ESP32-S2, ESP32-S3, ESP32-C3, ESP32-CAM
↳ Raspberry Pi Pico	MicroPython blocks for RP2040 — GPIO, ADC, I2C, SPI, PIO
↳ Raspberry Pi 4/5	Python blocks for GPIO, camera (picamera2), I2C sensors, SSH upload
AI/ML Integration	
↳ TensorFlow Lite Micro	On-device inference blocks — image classification, anomaly detection
↳ Voice Recognition	Keyword spotting blocks for ESP32-S3 (PSRAM + microphone)
↳ Data Collection	Sensor data logging blocks for ML model training datasets
Database & Cloud	
↳ Firebase Integration	Real-time database read/write blocks, user authentication
↳ Supabase Support	PostgreSQL-backed data storage, REST API blocks
↳ Cloud Project Sync	Save/load projects to cloud, version history, sharing
Dashboard & Analytics	

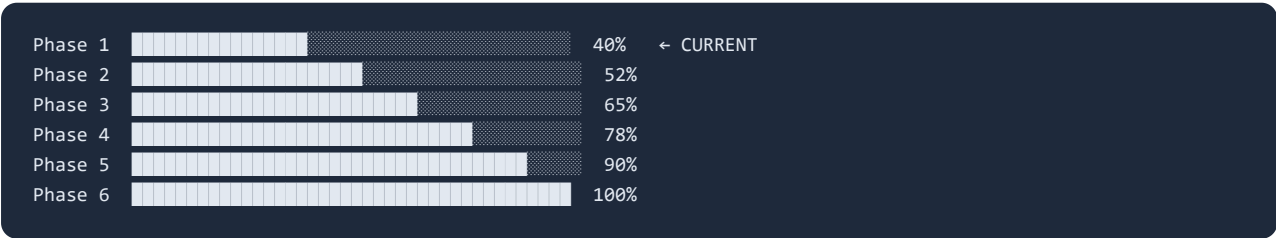
Deliverable	Detail
↳ Real-time Dashboard	Live sensor data visualization with charts and gauges
↳ Data Export	CSV/JSON export of collected sensor data
Subscription System	Free/Pro/Enterprise tier management, license validation
Documentation	User guide, block reference, API documentation, tutorials
Performance Optimization	Bundle optimization, lazy loading, caching strategies

10. Milestone Deliverables

10.1 Master Milestone Table

Phase	Name	Timeline	Feature Scope	New Blocks	Cumulative Blocks	Completion	Key Deliverables
1	Foundation	Month 0	Core platform, Scratch mode, basic ESP32	182	182 / 317	40%	Scratch engine, basic hardware, MicroPython gen
2	Sensors + Upload	Month 1–2	Full sensor library, upload pipeline	40	222 / 317	52%	47 sensor blocks, Web Serial upload, Serial Monitor v2
3	Arduino + Actuators	Month 3–4	Arduino C++, full actuators	13	235 / 317	65%	Arduino generators, compile pipeline, actuator blocks
4	Displays + Motors	Month 5–6	Display output, motor control	15	250 / 317	78%	LCD, NeoPixel, L298N, DC motor
5	Comms + IoT	Month 7–9	Full IoT connectivity	50	300 / 317	90%	WiFi, BLE, HTTP, MQTT, Blynk, ThingSpeak
6	Multi-Board + AI/ML + DB	Month 10–12	RPi boards, AI/ML, database, cloud, docs	30+	350+ / 350+	100%	Multi-board, TF Lite, Firebase, dashboard, docs

10.2 Cumulative Progress Visualization



10.3 Acceptance Criteria Per Phase

Phase	Acceptance Gate
Phase 1	All 96 Scratch blocks execute correctly; 32 ESP32 blocks generate valid MicroPython; project save/load functional
Phase 2	All 47 sensor blocks produce valid MicroPython; code uploads and executes on ESP32 via Web Serial; serial monitor reads device output
Phase 3	All enabled blocks generate valid Arduino C++; Arduino CLI compiles without errors; compiled firmware flashes to ESP32
Phase 4	LCD displays text correctly; NeoPixel animations render; motors respond to speed/direction commands
Phase 5	ESP32 connects to WiFi and sends HTTP/MQTT data; Blynk dashboard reflects device state; ThingSpeak logs sensor data
Phase 6	ESP32-CAM captures images; RPi Pico and RPi 4/5 blocks generate valid code; TF Lite model runs on ESP32-S3; Firebase database reads/writes succeed; cloud project sync functional; documentation complete

11. Quality Assurance & Testing

11.1 Testing Strategy

Test Category	Scope	Method	Frequency
Block-Level Testing	Each block generates syntactically valid code	Automated code generation + syntax validation	Every block addition
Upload Verification	Generated code compiles and runs on real ESP32	Manual hardware testing on physical ESP32 dev boards	Per phase delivery
Cross-Browser Testing	Chrome 89+, Edge 89+	Manual + automated browser testing	Per phase delivery
Responsive Design	Desktop (1024px+), Tablet (768px+)	Manual viewport testing	Per phase delivery
Engine Integrity	Sprite engine, block interpreter, event bus	Unit tests + integration tests	Continuous
Code Generation Parity	MicroPython and Arduino C++ produce equivalent behavior	Side-by-side execution comparison on ESP32	Phase 3 onwards

11.2 Hardware Test Matrix

Hardware	Firmware	Test Scope	Phases
ESP32 DevKit v1	MicroPython 1.20+	All MicroPython blocks	1–6

Hardware	Firmware	Test Scope	Phases
ESP32 DevKit v1	Arduino Core 2.x	All Arduino C++ blocks	3–6
ESP32-CAM (AI Thinker)	Arduino Core 2.x	Camera blocks	6
ESP32-S3 DevKit	Both	Multi-board compatibility	6

11.3 Quality Gates

Each phase delivery must pass the following quality gates before acceptance:

1. **Code Generation Validity** — 100% of enabled blocks produce syntactically correct code in target language(s).
2. **Hardware Execution** — Generated code compiles, uploads, and executes correctly on target ESP32 hardware.
3. **UI Stability** — No console errors, no visual regressions, all UI components render correctly.
4. **Performance** — Page load under 3 seconds on standard broadband; workspace interaction at 60fps.
5. **Browser Compatibility** — Full functionality verified on Chrome 89+ and Edge 89+.

12. Risk Assessment

12.1 Risk Register

#	Risk	Impact	Probability	Mitigation
1	Web Serial API browser support changes	High	Low	Monitor Chromium release notes; maintain Arduino CLI fallback path
2	Blockly v12 breaking API changes	High	Medium	Pin Blockly version; wrap all Blockly calls through abstraction layer
3	ESP32 firmware fragmentation	Medium	Medium	Test against pinned firmware versions; document minimum firmware requirements
4	Arduino CLI compilation failures on server	Medium	Medium	Comprehensive error handling; pre-validate board/library dependencies
5	IoT service API changes (Blynk, ThingSpeak)	Medium	Medium	Abstract API calls; version-pin integrations; implement graceful degradation
6	PixiJS v8 performance on low-end devices	Low	Low	Implement Canvas fallback; optimize sprite batch rendering
7	Block count growth exceeds toolbox UX limits	Medium	Low	Implement category folding, block search, and favorites system

12.2 Dependencies

Dependency	Type	Risk Level	Fallback
Google Blockly v12	Core Engine	Critical	Version-pinned, no alternative
PixiJS v8	Rendering	High	Canvas 2D fallback
Web Serial API	Upload	High	Arduino CLI server-side upload
Arduino CLI	Compilation	Medium	MicroPython-only mode
Node.js / Express	Server	Low	Static hosting + client-side only

13. Appendix

13.1 File Structure Overview

```
TechyGuide/
├─ src/
│   └─ blocks/                                # Block definitions
│       └─ esp32/                             # 167 ESP32 hardware blocks
│           └─ core/                         # Program structure, pins
│               └─ inputs/                   # Buttons, switches, touch
│                   └─ sensors/              # 14 sensor subcategories
│                       └─ actuators/        # Servo, relay, LED, buzzer
│                           └─ displays/     # LCD, NeoPixel
│                               └─ motors/    # L298N, DC motors
│                                   └─ comms/  # Serial, BT, WiFi, HTTP, MQTT
│                                       └─ dabble/ # Dabble app blocks
│   └─ scratch/                             # 96 Scratch animation blocks
│       └─ standard/                       # 54 standard programming blocks
│   └─ generators/                         # Code generators (47 files)
│       └─ micropython/                   # 19 MicroPython generators
│           └─ arduino/                   # 19 Arduino C++ generators
│               └─ core/                  # 9 framework generators
│   └─ engine/                             # Runtime engine (5 files)
│       └─ blockInterpreter.js            # Block execution engine
│       └─ stageRenderer.js              # PixiJS stage rendering
│       └─ spriteEngine.js               # Sprite lifecycle management
│       └─ spriteStore.js                # Sprite state management
│       └─ eventBus.js                   # Event dispatch system
│   └─ components/                        # UI components (14)
│       └─ modeSwitcher.js
│       └─ serialMonitor.js
│       └─ connectModal.js
│       └─ uploadPanel.js
│       └─ blockSearch.js
│       └─ spritePanel.js
│       └─ libraryManager.js
│       └─ subscriptionModal.js
│       └─ ...
│   └─ upload/                            # Upload system
│       └─ webSerial.js                  # Web Serial API interface
│           └─ arduinoCli.js             # Arduino CLI integration
│   └─ assets/                            # Sprites, backdrops, sounds
├─ server/                                # Node.js Express server
│   └─ compile.js                        # Arduino compilation endpoint
├─ webpack.config.js                     # Webpack 5 configuration
├─ package.json
└─ BLUEPRINT.md                          # ← This document
```

13.2 Glossary

Term	Definition
Block	A visual programming element that represents a code statement or expression
Blockly	Google's open-source library for building visual block programming editors
ESP32	A low-cost, low-power SoC with integrated Wi-Fi and Bluetooth, by Espressif

Term	Definition
Generator	A module that converts visual blocks into target language source code
MicroPython	A lean implementation of Python 3 optimized for microcontrollers
PixiJS	A fast 2D rendering engine using WebGL with Canvas fallback
Scratch Mode	The animation-focused workspace with sprite-based visual programming
Board Mode	The hardware-focused workspace with ESP32-specific programming blocks
Stage	The visual output area (480×360) where sprites are rendered and animated
Sprite	An animated visual object on the stage with costumes, position, and behaviors
Web Serial API	A browser API enabling direct serial communication with USB devices
Arduino CLI	Command-line tool for compiling and uploading Arduino sketches
MQTT	Lightweight messaging protocol for IoT device communication
Blynk	An IoT platform for device control and data visualization
ThingSpeak	An IoT analytics platform for sensor data aggregation and visualization
NeoPixel	Addressable RGB LED strips using the WS2812B chipset
L298N	A dual H-bridge motor driver IC for controlling DC and stepper motors
Dabble	A mobile app for controlling ESP32 via Bluetooth with virtual instruments

TechyGuide — Visual Programming Platform for ESP32 & IoT Education

Version 1.0.0 · June 2026 · Phase 1 Delivered

This document constitutes the official project blueprint and technical specification. Full-featured development with AI/ML integration, multi-board support, and cloud database is estimated to take approximately 12 months.

© 2026 TechyGuide. All rights reserved.